Exercise 1: Inventory Management System

1. Understanding the Problem

Why Data Structures and Algorithms are Essential?

In a warehouse inventory system, thousands of products are stored and managed. Efficient data structures and algorithms help in:

- Fast product searching.
- Efficient insertion and deletion.
- Quick inventory updates.
- Better memory utilization.
- Improved system performance for large datasets.

Suitable Data Structures

Data Structure	Usage
ArrayList	Stores products dynamically
HashMap	Fast search using Product ID
LinkedList	Frequent insertions/deletions
TreeMap	Sorted product records

For this system, **HashMap** is preferred because searching, updating, and deleting products can be performed in O(1) time.

---

2. Product Class

```
public class Product
   private int productId;
   private String productName;
   private int quantity;
   private double price;
   public Product(int productId,
                  String productName,
                  int quantity,
                  double price) {
      this.productId = productId;
      this.productName = productName;
      this.quantity = quantity;
      this.price = price;
   }
   public int getProductId() {
```

```
        return productId;
    }
    public String getProductName() {
        return productName;
    }
    public int getQuantity() {
        return quantity;
    }
    public double getPrice() {
        return price;
    }
    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }
    public void setPrice(double price) {
        this.price = price;
    }
    @Override
    public String toString() {
        return "ID: " + productId +
               ", Name: " + productName +
               ", Quantity: " + quantity +
               ", Price: " + price;
    }
}
```

---

3. Inventory Management Class

```
import java.util.HashMap;
public class InventoryManager {
    HashMap<Integer, Product> inventory =
            new HashMap<>();
    public void addProduct(Product product) {
        inventory.put(
                product.getProductId(),
                product);
        System.out.println("Product Added");
    }
    public void updateProduct(
            int productId,
            int quantity,
            double price) {
        Product product =
                inventory.get(productId);
        if(product != null) {
```

```
            product.setQuantity(quantity);
            product.setPrice(price);
            System.out.println(
                  "Product Updated");
        }
        else {
            System.out.println(
                  "Product Not Found");
        }
    }
    public void deleteProduct(
          int productId) {
        inventory.remove(productId);
        System.out.println(
              "Product Deleted");
    }
    public void displayProducts() {

        for(Product product :
              inventory.values()) {

        System.out.println(product);
        }
    }
}
```

---

4. Main Class

```
public class InventoryTest {
    public static void main(String[] args) {
        InventoryManager manager =
              new InventoryManager();
        Product p1 =
              new Product(
                    101,
                    "Laptop",
                    10,
                    50000);
        Product p2 =
              new Product(
                    102,
                    "Mouse",
                    50,
                    500);
        manager.addProduct(p1);
        manager.addProduct(p2);
```

```
        System.out.println("\nInventory");
        manager.displayProducts();
        manager.updateProduct(
                101,
                15,
                52000);
        System.out.println(
                "\nAfter Update");
        manager.displayProducts();
        manager.deleteProduct(102);
        System.out.println(
                "\nAfter Deletion");
        manager.displayProducts();
    }
}
```

Output

```
Product Added
Product Added

Inventory

ID: 101, Name: Laptop, Quantity: 10, Price: 50000.0
ID: 102, Name: Mouse, Quantity: 50, Price: 500.0

Product Updated

After Update

ID: 101, Name: Laptop, Quantity: 15, Price: 52000.0
ID: 102, Name: Mouse, Quantity: 50, Price: 500.0

Product Deleted

After Deletion

ID: 101, Name: Laptop, Quantity: 15, Price: 52000.0
```

---

5. Time Complexity Analysis

Operation	HashMap Complexity
Add Product	O(1)
Search Product	O(1)
Update Product	O(1)
Delete Product	O(1)
Display Products	O(n)

Where **n = number of products**.

---

6. Optimization Discussion

Current Optimization

- Using HashMap for constant-time access.

Further Improvements

- Use TreeMap for sorted inventory.
- Use Database Indexing for millions of records.
- Implement Caching for frequently searched products.
- Use ConcurrentHashMap for multi-user environments.

Exercise 2: E-commerce Platform Search Function

1. Understanding Asymptotic Notation

What is Big O Notation?

Big O Notation measures the efficiency of an algorithm as the input size increases.

It helps us:

- Compare algorithms.
- Predict performance.
- Choose the best solution.

Common Complexities

O(1)	Constant Time
O(log n)	Logarithmic Time
O(n)	Linear Time
O(n log n)	Efficient Sorting
$O(n^2)$	Nested Loops

Search Cases

Best Case

Item found at first position.

Time Complexity = O(1)

Average Case

Item found somewhere in the middle.

Time Complexity = O(n)

Worst Case

Item found at last position or not found.

Time Complexity = O(n)

---

2. Product Class

```
public class Product {
   private int productId;
   private String productName;
   private String category;
   public Product(int productId,
                  String productName,
                  String category) {
      this.productId = productId;
      this.productName = productName;
      this.category = category;
   }
   public int getProductId() {
      return productId;
   }
   public String getProductName() {
      return productName;
```

```
    }
    public String getCategory() {
        return category;
    }
    @Override
    public String toString() {
        return "ID : " + productId +
                " Name : " + productName +
                " Category : " + category;
    }
}
```

---

3. Linear Search Implementation

Search Class

```
public class LinearSearch {
    public static Product search(
            Product[] products,
            String targetName) {
        for(Product product : products) {
            if(product.getProductName()
                    .equalsIgnoreCase(targetName)) {
                return product;
            }
        }
        return null;
    }
}
```

---

4. Binary Search Implementation

Product Array Must Be Sorted

```
import java.util.Arrays;
import java.util.Comparator;
public class BinarySearch {
    public static Product search(
            Product[] products,
            String targetName) {
        int left = 0;
        int right = products.length - 1;
        while(left <= right) {
            int mid =
                    (left + right) / 2;
            int result =
```

```
                products[mid]
                .getProductName()
                .compareToIgnoreCase(targetName);
            if(result == 0) {
                return products[mid];
            }
            else if(result < 0) {
                left = mid + 1;
            }
            else {
                right = mid - 1;
            }
        }
        return null;
    }
}
```

---

5. Main Program

```
import java.util.Arrays;
import java.util.Comparator;
public class SearchTest {
    public static void main(String[] args) {
        Product[] products = {
                new Product(
                        101,
                        "Laptop",
                        "Electronics"),
                new Product(
                        102,
                        "Mobile",
                        "Electronics"),
                new Product(
                        103,
                        "Shoes",
                        "Fashion"),
                new Product(
                        104,
                        "Watch",
                        "Accessories")
        };
        System.out.println(
                "LINEAR SEARCH");
        Product result1 =
                LinearSearch.search(
                    products,
```

```
                "Shoes");
        if(result1 != null)
            System.out.println(result1);
        Arrays.sort(
                products,
                Comparator.comparing(
                        Product::getProductName));
        System.out.println(
                "\nBINARY SEARCH");
        Product result2 =
                BinarySearch.search(
                        products,
                        "Watch");
        if(result2 != null)
            System.out.println(result2);
    }
}
```

Output

```
LINEAR SEARCH

ID : 103
Name : Shoes
Category : Fashion

BINARY SEARCH

ID : 104
Name : Watch
Category : Accessories
```

---

6. Time Complexity Analysis

Linear Search

Case	Complexity
Best	O(1)
Average	O(n)
Worst	O(n)

Binary Search

Case	Complexity
Best	O(1)
Average	O(log n)
Worst	O(log n)

---

7. Comparison

Feature	Linear Search	Binary Search
Data Requirement	Unsorted	Sorted
Best Case	O(1)	O(1)
Average Case	O(n)	O(log n)
Worst Case	O(n)	O(log n)
Large Data Performance	Slow	Fast

---

8. Which Algorithm is Better?

Linear Search

Use when:

- Data is small.
- Data is unsorted.

Binary Search

Use when:

- Data is large.
- Data is sorted.
- High search performance is required.

For an **E-commerce Platform**, **Binary Search is more suitable** because product catalogs can contain thousands or millions of products and searching must be fast.

Exercise 3: Sorting Customer Orders

1. Understanding Sorting Algorithms

Sorting is the process of arranging data in ascending or descending order.

Bubble Sort

- Compares adjacent elements.
- Swaps them if they are in the wrong order.
- Repeats until the array is sorted.

Insertion Sort

- Builds the sorted array one element at a time.
- Efficient for small datasets.

Quick Sort

- Uses a pivot element.
- Partitions the array around the pivot.
- Recursively sorts sub-arrays.

Merge Sort

- Divides the array into halves.
- Sorts and merges them.
- Very efficient for large datasets.

---

2. Order Class

```
public class Order {
   private int orderId;
   private String customerName;
   private double totalPrice;
   public Order(int orderId,
                String customerName,
                double totalPrice) {
      this.orderId = orderId;
      this.customerName = customerName;
      this.totalPrice = totalPrice;
   }
   public double getTotalPrice() {
      return totalPrice;
   }
   @Override
```

```
    public String toString() {
        return "Order ID: " + orderId +
               ", Customer: " + customerName +
               ", Total Price: " + totalPrice;
    }
}
```

---

3. Bubble Sort Implementation

```
public class BubbleSort
    public static void sort(Order[] orders) {
        int n = orders.length;
        for(int i = 0; i < n - 1; i++) {
            for(int j = 0; j < n - i - 1; j++) {
                if(orders[j].getTotalPrice() >
                   orders[j + 1].getTotalPrice()) {
                    Order temp = orders[j];
                    orders[j] = orders[j + 1];
                    orders[j + 1] = temp;
                }
            }
        }
    }
}
```

---

4. Quick Sort Implementation

```
public class QuickSort {
    public static void quickSort(
            Order[] orders,
            int low,
            int high) {
        if(low < high) {
            int pi =
                partition(
                    orders,
                    low,
                    high);
            quickSort(
                orders,
                low,
                pi - 1);
            quickSort(
                orders,
                pi + 1,
```

```
            high);
        }
    }
    private static int partition(
            Order[] orders,
            int low,
            int high) {
        double pivot =
            orders[high].getTotalPrice();
        int i = low - 1;
        for(int j = low;
            j < high;
            j++) {
            if(orders[j]
                    .getTotalPrice()
                    < pivot) {
                i++;
                Order temp = orders[i];
                orders[i] = orders[j];
                orders[j] = temp;
            }
        }
        Order temp = orders[i + 1];
        orders[i + 1] = orders[high];
        orders[high] = temp;
        return i + 1;
    }
}
```

---

5. Main Program

```
public class SortingTest {
    public static void main(String[] args) {
        Order[] orders = {
            new Order(
                101,
                "Rahul",
                5000),
            new Order(
                102,
                "Priya",
                12000),
            new Order(
                103,
                "Anil",
                3000),
```

```
            new Order(
                104,
                "Sneha",
                8000)
        };
        System.out.println(
            "Before Sorting");
        for(Order order : orders)
            System.out.println(order);
        BubbleSort.sort(orders);
        System.out.println(
            "\nAfter Bubble Sort");
        for(Order order : orders)
            System.out.println(order);
        QuickSort.quickSort(
            orders,
            0,
            orders.length - 1);
        System.out.println(
            "\nAfter Quick Sort");
        for(Order order : orders)
            System.out.println(order);
    }
}
```

Output

```
Before Sorting

Order ID: 101, Customer: Rahul, Total Price: 5000
Order ID: 102, Customer: Priya, Total Price: 12000
Order ID: 103, Customer: Anil, Total Price: 3000
Order ID: 104, Customer: Sneha, Total Price: 8000

After Bubble Sort

Order ID: 103, Customer: Anil, Total Price: 3000
Order ID: 101, Customer: Rahul, Total Price: 5000
Order ID: 104, Customer: Sneha, Total Price: 8000
Order ID: 102, Customer: Priya, Total Price: 12000

After Quick Sort

Order ID: 103, Customer: Anil, Total Price: 3000
Order ID: 101, Customer: Rahul, Total Price: 5000
```

Order ID: 104, Customer: Sneha, Total Price: 8000
Order ID: 102, Customer: Priya, Total Price: 12000

---

6. Time Complexity Analysis

Bubble Sort

Case	Complexity
Best	O(n)
Average	$O(n^2)$
Worst	$O(n^2)$

Quick Sort

Case	Complexity
Best	O(n log n)
Average	O(n log n)
Worst	$O(n^2)$

---

7. Comparison

Feature	Bubble Sort	Quick Sort
Speed	Slow	Fast
Large Data	Poor	Excellent
Average Complexity	$O(n^2)$	O(n log n)
Memory Usage	Low	Low
Practical Usage	Rare	Widely Used

---

8. Why Quick Sort is Preferred?

1. Faster for large datasets.
2. Average complexity is O(n log n).
3. Uses divide-and-conquer strategy.
4. Efficient memory utilization.

Exercise 4: Employee Management System

1. Understanding Array Representation

What is an Array?

An array is a collection of elements stored in contiguous memory locations.

Example

```
Index:   0     1     2     3
        ----------------------
Data : | E1 | E2 | E3 | E4 |
        ----------------------
```

Each element can be accessed using its index.

Advantages of Arrays

1. Fast access using index.
2. Simple implementation.
3. Efficient memory usage.
4. Suitable for fixed-size data.

---

2. Employee Class

```
public class Employee {
   private int employeeId;
   private String name;
   private String position;
   private double salary;
   public Employee(int employeeId,
                   String name,
                   String position,
                   double salary) {
      this.employeeId = employeeId;
      this.name = name;
      this.position = position;
      this.salary = salary;
   }
   public int getEmployeeId() {
      return employeeId;
   }
   @Override
```

```
    public String toString() {
        return "ID: " + employeeId +
               ", Name: " + name +
               ", Position: " + position +
               ", Salary: " + salary;
    }
}
```

---

3. Employee Management System

```
public class EmployeeManagement {
    private Employee[] employees;
    private int count;
    public EmployeeManagement(int size) {
        employees = new Employee[size];
        count = 0;
    }
    // Add Employee
    public void addEmployee(Employee employee) {
        if(count < employees.length) {
            employees[count] = employee;
            count++;
            System.out.println(
                   "Employee Added");
        }
        else {
            System.out.println(
                   "Array Full");
        }
    }
    // Search Employee
    public Employee searchEmployee(
           int employeeId) {
        for(int i = 0; i < count; i++) {
            if(employees[i]
                   .getEmployeeId()
                   == employeeId) {
                return employees[i];
            }
        }
        return null;
    }
    // Traverse Employees
    public void displayEmployees() {
        for(int i = 0; i < count; i++) {
```

```
            System.out.println(
                    employees[i]);
            }
        }
        // Delete Employee
        public void deleteEmployee(
                int employeeId) {
            int index = -1;
            for(int i = 0; i < count; i++) {
                if(employees[i]
                        .getEmployeeId()
                        == employeeId) {
                    index = i;
                    break;
                }
            }
            if(index != -1) {
                for(int i = index;
                    i < count - 1;
                    i++) {
                    employees[i] =
                            employees[i + 1];
                }
                employees[count - 1] = null;
                count--;
                System.out.println(
                        "Employee Deleted");
            }
            else {
                System.out.println(
                        "Employee Not Found");
            }
        }
    }
```

---

4. Main Program

```
public class EmployeeTest {
    public static void main(String[] args) {
        EmployeeManagement manager =
                new EmployeeManagement(10);
        manager.addEmployee(
                new Employee(
                        101,
                        "Rahul",
                        "Manager",
```

```
                50000));
        manager.addEmployee(
                new Employee(
                        102,
                        "Priya",
                        "Developer",
                        40000));
        manager.addEmployee(
                new Employee(
                        103,
                        "Anil",
                        "Tester",
                        35000));
        System.out.println(
                "\nAll Employees");
        manager.displayEmployees();
        System.out.println(
                "\nSearching Employee 102");
        Employee employee =
                manager.searchEmployee(102);
        if(employee != null) {
           System.out.println(employee);
        }
        manager.deleteEmployee(102);
        System.out.println(
                "\nAfter Deletion");
        manager.displayEmployees();
    }
}
```

Output

```
Employee Added
Employee Added
Employee Added

All Employees

ID: 101, Name: Rahul, Position: Manager, Salary: 50000.0
ID: 102, Name: Priya, Position: Developer, Salary: 40000.0
ID: 103, Name: Anil, Position: Tester, Salary: 35000.0

Searching Employee 102

ID: 102, Name: Priya, Position: Developer, Salary: 40000.0
```

Employee Deleted

After Deletion

ID: 101, Name: Rahul, Position: Manager, Salary: 50000.0
ID: 103, Name: Anil, Position: Tester, Salary: 35000.0

---

5. Time Complexity Analysis

Operation	Complexity
Add Employee	O(1)
Search Employee	O(n)
Traverse Employees	O(n)
Delete Employee	O(n)

Where **n = number of employees**.

---

6. Limitations of Arrays

1. Fixed size.
2. Insertion in middle is costly.
3. Deletion requires shifting elements.
4. Memory may be wasted.
5. Not suitable for highly dynamic data.

---

7. When Should Arrays Be Used?

Use arrays when:

- Data size is known.
- Frequent indexing is required.
- Memory efficiency is important.
- Insertions and deletions are less frequent.

Exercise 5: Task Management System (Linked List)

1. Understanding Linked Lists

A Linked List is a dynamic data structure where elements are stored in nodes.

Each node contains:

- Data
- Reference (Link) to the next node

Types of Linked Lists

1. Singly Linked List

Task1 -> Task2 -> Task3 -> NULL

Each node points to the next node.

2. Doubly Linked List

NULL <- Task1 <-> Task2 <-> Task3 -> NULL

Each node contains:

- Previous pointer
- Next pointer

Advantages of Linked Lists

- Dynamic size.
- Easy insertion and deletion.
- No memory wastage.
- No shifting of elements required.

---

2. Task Class

```
public class Task {
   int taskId;
   String taskName;
   String status
   public Task(int taskId,
               String taskName,
               String status) {
      this.taskId = taskId;
      this.taskName = taskName;
```

```
        this.status = status;
    }
    @Override
    public String toString() {
        return "Task ID: " + taskId +
               ", Name: " + taskName +
               ", Status: " + status;
    }
}
```

---

3. Node Class

```
class Node {
    Task task;
    Node next;
    public Node(Task task) {
        this.task = task;
        this.next = null;
    }
}
```

---

4. Singly Linked List Implementation

```
public class TaskLinkedList {
    private Node head;
    // Add Task
    public void addTask(Task task) {
        Node newNode = new Node(task);
        if(head == null) {
            head = newNode;
        }
        else {
            Node temp = head;
            while(temp.next != null) {
                temp = temp.next;
            }
            temp.next = newNode;
        }
        System.out.println("Task Added");
    }
    // Search Task
    public Task searchTask(int taskId) {
        Node temp = head;
        while(temp != null) {
            if(temp.task.taskId == taskId) {
```

```
            return temp.task;
        }
        temp = temp.next;
    }
    return null;
}
// Display Tasks
public void displayTasks() {
    Node temp = head;
    while(temp != null) {
        System.out.println(temp.task);
        temp = temp.next;
    }
}
// Delete Task
public void deleteTask(int taskId) {
    if(head == null) {
        return;
    }
    if(head.task.taskId == taskId) {
        head = head.next;
        System.out.println("Task Deleted");
        return;
    }
    Node temp = head;
    while(temp.next != null &&
          temp.next.task.taskId != taskId) {
        temp = temp.next;
    }
    if(temp.next != null) {
        temp.next = temp.next.next
        System.out.println("Task Deleted");
    }
    else {
        System.out.println("Task Not Found");
    }
}
}
```

---

5. Main Program

```
public class TaskTest
    public static void main(String[] args) {
        TaskLinkedList list =
                new TaskLinkedList();
        list.addTask(
```

```
            new Task(
                    101,
                    "Design UI",
                    "Pending"))
        list.addTask(
             new Task(
                    102,
                    "Develop Backend",
                    "In Progress"));
        list.addTask(
             new Task(
                    103,
                    "Testing",
                    "Pending"));
        System.out.println(
             "\nAll Tasks");
        list.displayTasks()
        System.out.println(
             "\nSearch Task 102");
        Task task =
             list.searchTask(102);
        if(task != null) {
           System.out.println(task);
        }
        list.deleteTask(102);
        System.out.println(
             "\nAfter Deletion");
        list.displayTasks();
    }
}
```

Output

```
Task Added
Task Added
Task Added

All Tasks

Task ID: 101, Name: Design UI, Status: Pending
Task ID: 102, Name: Develop Backend, Status: In Progress
Task ID: 103, Name: Testing, Status: Pending

Search Task 102

Task ID: 102, Name: Develop Backend, Status: In Progress
```

Task Deleted

After Deletion

Task ID: 101, Name: Design UI, Status: Pending
Task ID: 103, Name: Testing, Status: Pending

---

6. Time Complexity Analysis

Operation	Complexity
Add Task	O(n)
Search Task	O(n)
Traverse Tasks	O(n)
Delete Task	O(n)

Where **n = number of tasks**.

---

7. Advantages of Linked Lists Over Arrays

Linked List	Array
Dynamic size	Fixed size
Easy insertion/deletion	Costly insertion/deletion
No shifting required	Shifting required
Better for dynamic data	Better for indexing

---

8. When to Use Linked Lists?

Use Linked Lists when:

- Data size changes frequently.
- Frequent insertion/deletion operations are required.
- Memory allocation should be dynamic.

Exercise 6: Library Management System (Linear Search & Binary Search)

1. Understanding Search Algorithms

Searching is the process of finding a specific element in a collection of data.

Linear Search

Linear Search checks each element one by one until the required item is found.

Example

Books = [Java, Python, C++, DBMS]

Search = C++

Java → Not Found
Python → Not Found
C++ → Found

Characteristics

- Works on sorted and unsorted data.
- Easy to implement.
- Suitable for small datasets.

Binary Search

Binary Search repeatedly divides the sorted list into two halves.

Example

Books = [C++, DBMS, Java, Python]

Search = Java

Middle = DBMS
Java > DBMS

Search Right Half

Middle = Java

Found

Characteristics

- Requires sorted data.
- Much faster than Linear Search.
- Suitable for large datasets.

---

2. Book Class

```
public class Book {

   private int bookId;
   private String title;
   private String author;

   public Book(int bookId,
               String title,
               String author) {

      this.bookId = bookId;
      this.title = title;
      this.author = author;
   }

   public int getBookId() {
      return bookId;
   }

   public String getTitle() {
      return title;
   }

   public String getAuthor() {
      return author;
   }

   @Override
   public String toString() {

      return "Book ID: " + bookId +
            ", Title: " + title +
            ", Author: " + author;
   }
}
```

---

3. Linear Search Implementation

```
public class LinearSearch {

   public static Book search(Book[] books,
                             String title) {

      for(Book book : books) {

         if(book.getTitle()
               .equalsIgnoreCase(title)) {

            return book;
         }
      }

      return null;
   }
}
```

---

4. Binary Search Implementation

```
public class BinarySearch {

   public static Book search(Book[] books,
                             String title) {

      int left = 0;
      int right = books.length - 1;

      while(left <= right) {

         int mid = (left + right) / 2;

         int result =
            books[mid]
            .getTitle()
            .compareToIgnoreCase(title);

         if(result == 0) {

            return books[mid];
         }
         else if(result < 0) {

            left = mid + 1;
```

```
            }
            else {

                right = mid - 1;
            }
        }

        return null;
    }
}
```

---

5. Main Program

```
import java.util.Arrays;
import java.util.Comparator;

public class LibraryTest {

    public static void main(String[] args) {

        Book[] books = {

            new Book(
                    101,
                    "Java",
                    "James Gosling"),

            new Book(
                    102,
                    "Python",
                    "Guido van Rossum"),

            new Book(
                    103,
                    "DBMS",
                    "Korth"),

            new Book(
                    104,
                    "C++",
                    "Bjarne Stroustrup")
        };

        System.out.println(
                "Linear Search");
```

```
        Book result1 =
            LinearSearch.search(
                books,
                "Python");

        if(result1 != null) {

            System.out.println(result1);
        }

        Arrays.sort(
            books,
            Comparator.comparing(
                Book::getTitle));

        System.out.println(
                "\nBinary Search");

        Book result2 =
            BinarySearch.search(
                books,
                "Java");

        if(result2 != null) {

            System.out.println(result2);
        }
    }
}
```

Output

```
Linear Search

Book ID: 102,
Title: Python,
Author: Guido van Rossum

Binary Search

Book ID: 101,
Title: Java,
Author: James Gosling
```

---

6. Time Complexity Analysis

Linear Search

Case	Complexity
Best Case	O(1)
Average Case	O(n)
Worst Case	O(n)

Binary Search

Case	Complexity
Best Case	O(1)
Average Case	O(log n)
Worst Case	O(log n)

---

7. Comparison of Linear Search and Binary Search

Feature	Linear Search	Binary Search
Data Requirement	Sorted/Unsorted	Sorted Only
Best Case	O(1)	O(1)
Average Case	O(n)	O(log n)
Worst Case	O(n)	O(log n)
Large Data Performance	Slow	Fast
Implementation	Easy	Moderate

---

8. When to Use Each Algorithm

Use Linear Search When:

- Data size is small.
- Data is unsorted.
- Simplicity is preferred.

Use Binary Search When:

- Data is sorted.
- Dataset is large.
- High performance is required.

---

9. Conclusion

For a Library Management System:

- Linear Search is suitable for small libraries.
- Binary Search is preferred for large libraries with sorted book records because it provides significantly faster search performance.

Exercise 7: Financial Forecasting (Recursion)

1. Understanding Recursive Algorithms

What is Recursion?

Recursion is a programming technique where a function calls itself to solve a smaller version of the same problem.

A recursive function contains:

1. **Base Case** – stops recursion.
2. **Recursive Case** – function calls itself.

Example

```
public static int factorial(int n){

   if(n == 0)
      return 1;

   return n * factorial(n - 1);
}
```

Working

factorial(5)

5 × factorial(4)
4 × factorial(3)
3 × factorial(2)

2 × factorial(1)
1 × factorial(0)

Result = 120

Why Recursion is Useful in Financial Forecasting

Financial forecasting often involves predicting future values based on current values and growth rates.

Formula:

Future Value = Present Value × (1 + Growth Rate)

Using recursion, we can calculate the value year after year.

---

2. Recursive Method for Financial Forecasting

Assume:

- Initial Investment = ₹10,000
- Growth Rate = 10%
- Forecast Period = 5 Years

Recursive Formula

FV(n) = FV(n-1) × (1 + Growth Rate)

Base Case

FV(0) = Present Value

---

3. Implementation

```
public class FinancialForecast {

   public static double forecast(
         double presentValue,
         double growthRate,
         int years) {

      if(years == 0) {

         return presentValue;
      }
```

```
        return forecast(
                presentValue,
                growthRate,
                years - 1)
                * (1 + growthRate);
    }

    public static void main(String[] args) {

        double presentValue = 10000;

        double growthRate = 0.10;

        int years = 5;

        double futureValue =
                forecast(
                        presentValue,
                        growthRate,
                        years);

        System.out.println(
                "Future Value = " +
                futureValue);
    }
}
```

Output

Future Value = 16105.1

Step-by-Step Calculation

Year 0 = 10000

Year 1 = 10000 × 1.10
= 11000

Year 2 = 11000 × 1.10
= 12100

Year 3 = 12100 × 1.10
= 13310

Year 4 = 13310 × 1.10
= 14641

Year 5 = 14641 × 1.10
= 16105.1

Alternative Recursive Version

```
public class Forecast {

   static double predict(
         double value,
         int years) {

      if(years == 0)
         return value;

      return predict(
            value * 1.08,
            years - 1);
   }

   public static void main(String[] args) {

      System.out.println(
            predict(
                  5000,
                  3));
   }
}
```

Time Complexity Analysis

Recursive Solution

forecast(value, rate, n)

The function executes once for each year.

T(n) = T(n-1) + O(1)

Therefore:

Time Complexity = O(n)

where **n = number of years**.

Space Complexity

Because each recursive call is stored in the call stack:

Space Complexity = O(n)

---

4. Optimization Techniques

Problem with Recursion

For very large values of **n**:

- Many recursive calls are generated.
- Stack memory usage increases.
- May cause StackOverflowError.

Optimization 1: Iterative Approach

```
public static double forecast(
        double value,
        double rate,
        int years) {

    for(int i = 1;
        i <= years;
        i++) {

        value =
            value * (1 + rate);
    }

    return value;
}
```

Complexity

Time Complexity = O(n)
Space Complexity = O(1)

This is better than recursion.

Optimization 2: Dynamic Programming (Memoization)

Store previously calculated values.

double[] dp = new double[years + 1];

Avoids repeated calculations.

Optimization 3: Mathematical Formula

Compound Growth Formula:

Future Value =
Present Value × (1 + Rate)^Years

Java:

```
double futureValue =
        presentValue *
        Math.pow(
                1 + rate,
                years);
```

Complexity

Time Complexity = O(1)

Fastest approach.

Comparison of Approaches

Method	Time Complexity	Space Complexity
Recursion	O(n)	O(n)
Iteration	O(n)	O(1)
Dynamic Programming	O(n)	O(n)
Mathematical Formula	O(1)	O(1)

Conclusion

For Financial Forecasting:

- Recursion is easy to understand and implement.
- Iteration is more memory efficient.
- Mathematical formula using Math.pow() is the fastest and most practical solution for large-scale forecasting systems.
